Use-Case Diagrams

Capturing functional requirements & user interactions

24 Aug 2025

Agenda

- Introduction & Definition
- Notation: Actors, Use Cases & System Boundary
- Relationships & Scenarios
- Drawing Steps & Guidelines
- Examples & Case Study
- Common Mistakes & Best Practices
- Workshop & Homework

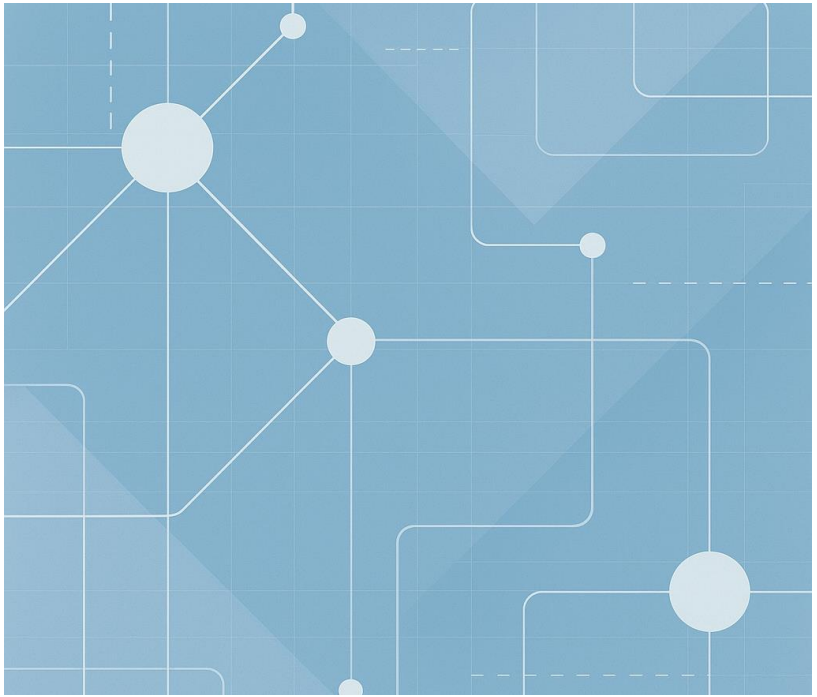
What is a Use-Case Diagram?

Use-case diagrams document the behaviour of a system from the user's point of view.

Each oval represents a task that must be supported by the system – a "use case" – while stick figures depict ***actors***, the external roles (people or systems) who interact with those tasks.

A use-case diagram shows not one use case but all the use cases for the system, capturing what the system must do – not how it will be implemented.

Use-case modelling helps with capturing requirements, planning iterations of development and validating a system by providing an intuitive picture that can be discussed with customers who may not know UML.



[3] [4]

Purpose & Benefits

Purpose

- Document user tasks and define the system’s boundaries
- Capture functional requirements and actor interactions
- Plan development iterations and manage risk
- Validate the system with stakeholders

Benefits

- Provide an intuitive picture for customers and non-technical stakeholders
- Create a shared understanding of roles and tasks among developers and customers
- Emphasise the user perspective and identify beneficiaries
- Support requirement capture and iteration planning

[5] [6]

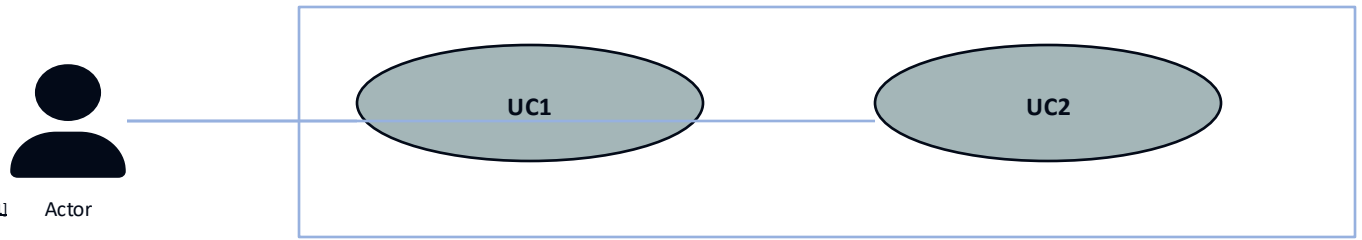
Actors & Use Cases

Actors

- Represent a role that someone or something plays – not a specific individual
- Identify beneficiaries who gain value from the use case
- One person may play multiple roles; identify each role distinctly
- Actors may be humans or systems; devices are not actors, and whether to show an external system depends on pragmatism
- Roles can be specialised via generalisation relationships

Use Cases

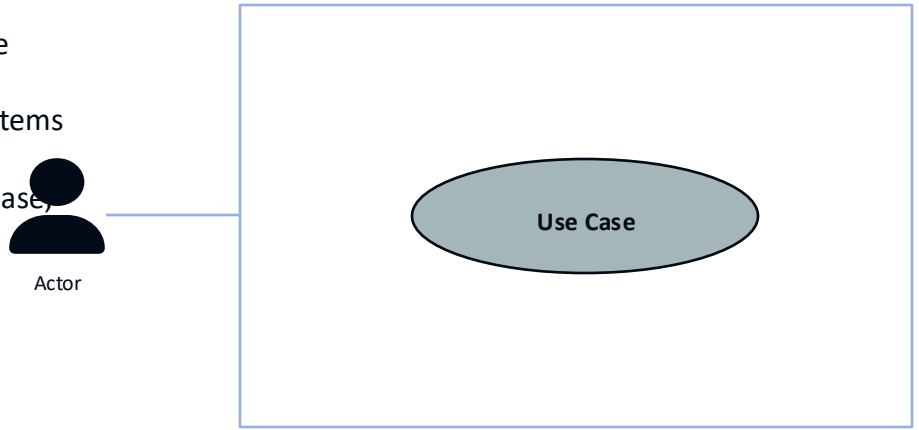
- Describe a sequence of actions that yields an observable result for an actor
- Group multiple scenarios (alternative flows) that achieve the same goal
- Triggered by actors and reflect user goals; may involve multiple actors
- Can reuse behaviour (include), separate variants (extend) or specialise (generalisation)
- Text descriptions list scenarios and alternative flows for completeness



[7] [8] [9] [10] [11]

System Boundary & Communication Links

• System Boundary: an optional rectangular box labelled with the system name that separates the system from its environment.
• Use it to organise complex systems or to show separate subsystems on different diagrams.
• Communication Link: a solid line connecting an actor to a use case, representing message exchange.
• Everything outside the boundary belongs to actors or other systems.



[12] [13]

Relationships

Association

Connects an actor to a use case to show participation
One actor may be linked to several use cases and vice versa
Undirected; they do not imply sequence or control

Include

Factor out common behaviour reused by multiple use cases
Source use case contains the behaviour of the included use case in all scenarios
Arrow points from the source to the included use case (<<include>>)

Extend

Separate variant or exceptional behaviour from the main case
Extending use case adds behaviour only when a condition is met
Arrow points from the extending case to the base case (<<extend>>)

Generalisation

Specialised use case or actor inherits behaviour of a more general one
Use for variations of whole tasks – not for optional behaviour
Arrow points from the specialised element to the general one

[14] [15] [16] [17] [18]

How to Draw a Use-Case Diagram

1. Identify actors by recognising the different roles and beneficiaries
2. Determine what use cases each actor needs from the system
3. Identify other interactions and obligations each actor expects
4. Prioritise use cases by value and risk; plan development iterations
5. Draw the system boundary to show what is inside the system
6. Define include, extend and generalisation relationships where appropriate
7. Write scenarios for each use case, covering normal and alternative flows
8. Review with stakeholders, refine and iterate

[19] [20] [21] [22] [23] [24] [25] [26] [27]

Scenarios vs Use-Cases

Definitions

- A use case groups together all the scenarios that achieve the same observable goal for an actor.
- A scenario is a single instance of a use case: a sequence of interactions between the actor and the system.
- Multiple scenarios may belong to one use case even if their steps differ, provided the end result is the same.

Example: Borrowing a Book

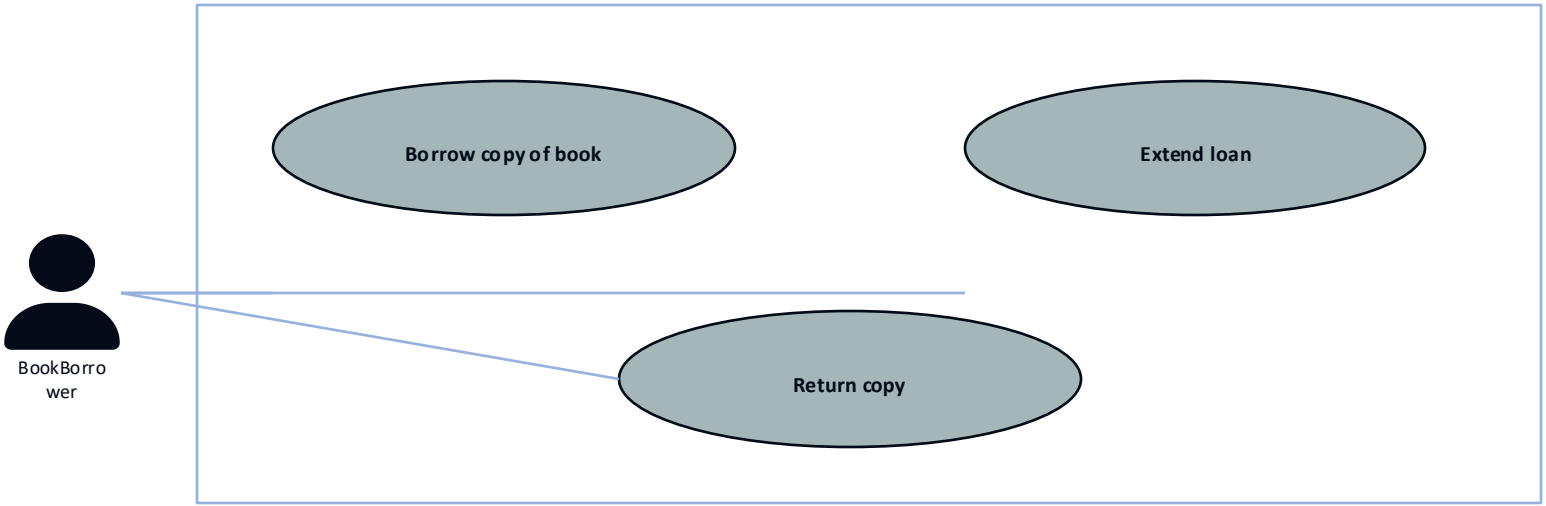
Scenario 1: Successful Borrowing

1. User requests to borrow a copy.
2. System checks reservation and loan limits.
3. System lends the book.

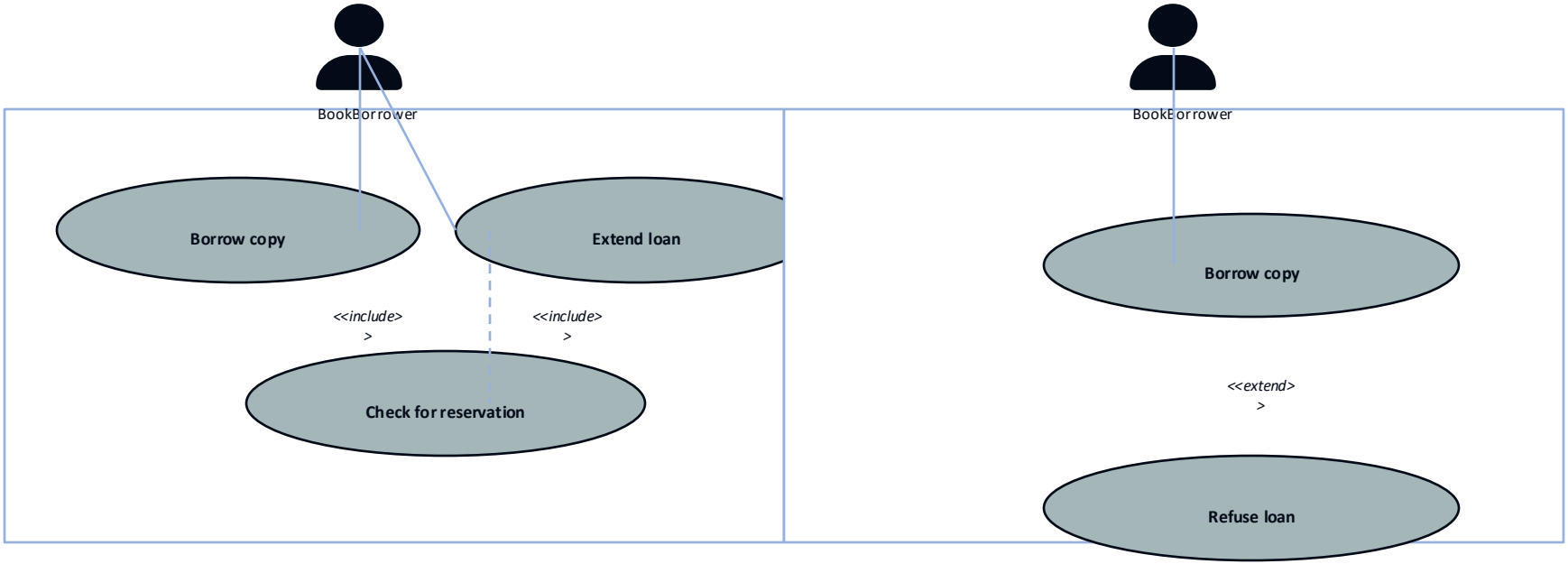
Scenario 2: Refusal (Maximum Items Reached)

1. User requests to borrow a copy.
2. System checks reservation and loan limits.
3. System refuses the loan because the maximum number of borrowed items is reached.

Example: Library System



Relationship Examples



[30] [31]

Common Mistakes & Best Practices

Mistakes

- Focusing solely on use cases can lead to a non object-oriented design
- Mistaking design decisions for requirements in use-case descriptions
- Missing requirements by relying only on actors and their use cases
- Overusing include, extend or generalisation – keep diagrams simple

Best Practices

- Model conceptual classes and CRC cards alongside use cases
- Keep diagrams simple; use include/extend only when beneficial
- Describe scenarios clearly; separate design decisions from requirements
- Collaborate with stakeholders and refine diagrams iteratively

[32] [33]

Workshop Activity

Form groups of 3–4 students and choose a scenario such as a library management or e-learning platform.

Identify all actors (e.g., students, librarians, administrators).
Identify the main use cases (search catalogue, borrow/return books, reserve seats, update catalog).
Draw the system boundary and connect actors to use cases using appropriate relationships.
Identify include/extend relationships and use case levels where appropriate.
Prepare to present your diagram to the class (time: 30 min).

[34] [35]

Homework Assignment

Design a use-case diagram for a new application (e.g., a food delivery app or university course registration system).

Identify at least two actors (e.g., customer, delivery person, restaurant, platform admin).
List the main use cases (browse menu, place order, pay, track order, manage menu, etc.).
Include at least one include and one extend relationship in your diagram.
Follow naming conventions and keep your diagram simple and clear.
Write a one-page explanation describing your assumptions and design decisions.
Submit your work as a PDF or image by 31 Aug 2025.

[36] [37] [38] [39]

Summary & Takeaways

- Use-case diagrams model system behaviour from the user’s point of view.
- A use case groups multiple scenarios that achieve the same observable goal.
- Actors represent roles (human or system) rather than specific individuals.
- Use include for common behaviour, extend for variant behaviour and generalisation for specialisations.
- Document scenarios and keep diagrams simple; plan iterations based on value and risk.
- Model classes in parallel and review diagrams with stakeholders to avoid pitfalls.

[40] [41] [42] [43] [44] [45]